UNIVERSIDAD NACIONAL DE INGENIERÍA

FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA

Ingeniería de Ciberseguridad

Chasky: DPSS de Detección en Tiempo Real de Phishing en Correos Electrónicos usando Transformers (DistilBERT) en Infraestructura GCP

Memoria Descriptiva Final

Integrantes:

Espinoza Soncco, Emerson Aldair
Minaya Diaz, Jose Miguel
Palacios Yarleque, Jerson Andres
Villegas Noblecilla, Luis Javier

Lima, Perú
2025

Chasky: Memoria Descriptiva del Software

Espinoza Soncco Emerson Aldair, Minaya Diaz Jose Miguel, Palacios Yarleque Jerson Andres, Villegas Noblecilla Luis Javier

CONTENTS

I. Introducción y Antecedentes 2
I-A. Contexto y Problemática . . . 2

II. Objetivos del Proyecto 2
II-A. Objetivo General . . . 2
II-B. Objetivos Específicos . . . 2

III. Descripción General del Software 2
III-A. Perspectiva del Producto . . . 2
III-B. Características del Usuario Final . . . 2

IV. Requerimientos de Software 2
IV-A. RF-01: Acceso a la Plataforma y Conexión de Cuenta . . . 2
IV-B. RF-02: Análisis Automático de Correos Electrónicos . . . 2
IV-C. RF-03: Generación de Notificaciones y Resúmenes . . . 2
IV-D. RF-04: Visualización de Resúmenes y Estadísticas . . . 3

V. Requisitos No Funcionales 3
V-A. Requisitos del Producto . . . 3
V-B. Requisitos de la Organización . . . 3
V-C. Requisitos Externos . . . 3

VI. Arquitectura del Sistema 3
VI-A. Principios y Diseño Arquitectónico . . . 3
VI-B. Vistas Arquitectónicas (Modelo 4+1) . . . 3
VI-B.1. Vista Lógica . . . 3
VI-B.2. Vista de Procesos . . . 4
VI-B.3. Vista de Desarrollo . . . 4
VI-B.4. Vista Física (Despliegue) . . . 4
VI-B.5. Vista de Casos de Uso (+1) . . . 4

VII. Modelos del Sistema 4
VII-A. Diseño Orientado a Objetos . . . 4
VII-A.1. Clases principales . . . 5
VII-A.2. Diagrama de Secuencia . . . 5

VIII. Conclusión 5

Referencias 5

Estudiantes de Ingeniería de Ciberseguridad, Universidad Nacional de Ingeniería, Lima, Perú. Correos: emerson.espinoza@uni.pe, miguel.minaya.d@uni.pe, jerson.palacios.y@uni.pe, luis.villegas.n@uni.pe

I. Introducción y Antecedentes

I-A. Contexto y Problemática

El correo electrónico es el principal vector para ciberataques (Butt et al., 2023; Fatima et al., 2024). El *phishing* genera pérdidas millonarias mediante el robo de credenciales (Albán et al., 2022). La sofisticación actual de los ataques evade los filtros estáticos tradicionales (Meléndez et al., 2024), creando la necesidad de soluciones basadas en IA (Johnson et al., 2020).

Los métodos tradicionales (SVM, Naive Bayes) logran alta precisión pero dependen de ingeniería manual de características. En contraste, los modelos Transformer como **DistilBERT** operan con texto crudo y ofrecen el equilibrio óptimo para sistemas en tiempo real: retienen el 97 % del rendimiento de BERT siendo un 40 % más ligeros y 60 % más rápidos (Alhuzali et al., 2025; Meléndez et al., 2024). Esta eficiencia justifica su elección para la infraestructura limitada de este proyecto.

II. Objetivos del Proyecto

II-A. Objetivo General

Desarrollar e implementar una Plataforma de Protección Dinámica (DPSS) de código abierto y bajo costo, capaz de detectar phishing en tiempo real en cuentas de Gmail utilizando modelos de Deep Learning (DistilBERT) sobre infraestructura de capa gratuita.

II-B. Objetivos Específicos

A partir del alcance definido, se establecen los siguientes objetivos específicos:

- Implementar un módulo de **autenticación segura** mediante OAuth 2.0 que permita el acceso a la API de Gmail sin comprometer las credenciales del usuario.
- Desarrollar un **motor de análisis automático** que clasifique continuamente los correos entrantes (Legítimo/Spam/Phishing) utilizando DistilBERT sin requerir ingeniería manual de características.
- Construir un **sistema de notificaciones y visualización** que alerte al usuario vía correo y presente estadísticas en un dashboard web accesible públicamente mediante túnel seguro (ngrok).
- Optimizar la arquitectura para operar en una instancia **e2-micro de GCP** con un costo operativo accesible, garantizando la sostenibilidad del proyecto.

III. Descripción General del Software

III-A. Perspectiva del Producto

Chasky se posiciona en el ecosistema de ciberseguridad no como un antivirus tradicional, sino como una capa adicional de protección inteligente especializada en la detección de phishing y spam en cuentas de Gmail, concebida como un Sistema de Salvaguarda Dinámica contra Phishing (DPSS).

A diferencia de las soluciones comerciales que implican altos costos y funcionan como cajas negras, Chasky es un proyecto completamente de código abierto que puede desplegarse sin costo alguno. Está diseñado especialmente para estudiantes, desarrolladores y pequeñas organizaciones, ofreciendo una protección avanzada basada en inteligencia artificial sin barreras económicas.

Gracias al empleo exclusivo del modelo DistilBERT ejecutado en la capa gratuita de Google Cloud Platform, Chasky constituye una barrera de defensa dinámica, transparente y fácilmente replicable, capaz de mantenerse eficaz frente a las tácticas cambiantes de los ciberdelincuentes sin depender de reglas estáticas ni de servicios externos de pago.

III-B. Características del Usuario Final

Corresponde a la persona (ej. estudiante, miembro de una pequeña organización) cuya cuenta de correo electrónico está siendo protegida. Se espera que este usuario no tenga conocimientos técnicos avanzados. Su interacción principal con el sistema se limita a conectar su cuenta a través de una interfaz web intuitiva y visualizar los reportes y alertas generados por la plataforma.

IV. Requerimientos de Software

En esta sección se describen en detalle las funcionalidades realmente implementadas en Chasky.

IV-A. RF-01: Acceso a la Plataforma y Conexión de Cuenta

Permite al usuario vincular de forma segura su cuenta de Gmail mediante OAuth 2.0.

Caso de uso principal: **Autenticación de Usuario (CU-01)**.

- **RS-1.1:** El sistema proporciona una interfaz web (Flask) con botón "Conectar con Google" que inicia el flujo OAuth 2.0.
- **RS-1.2:** Se solicitan únicamente los permisos `gmail.readonly` y `gmail.send` para leer correos y enviar alertas.
- **RS-1.3:** Los tokens de acceso y refresh se almacenan cifrados en variables de entorno del contenedor Docker.

IV-B. RF-02: Análisis Automático de Correos Electrónicos

El núcleo del sistema: monitoreo continuo y clasificación automática con DistilBERT.

- **RS-2.1:** Un hilo en segundo plano consulta periódicamente (cada 8–10 minutos) la bandeja de entrada mediante la Gmail API, descargando solo los mensajes no procesados previamente.
- **RS-2.2:** Se extrae asunto y cuerpo en texto plano. No se utilizan NLTK, SpaCy ni extracción manual de características.
- **RS-2.3:** Cada correo se clasifica en Legítimo, Spam o Phishing exclusivamente mediante DistilBERT (cargado una sola vez en memoria). No existe modelo híbrido ni Naive Bayes.

IV-C. RF-03: Generación de Notificaciones y Resúmenes

- **RS-3.1:** Cuando la confianza de phishing o spam supera el umbral configurado, se envía automáticamente una alerta por correo electrónico al propio usuario (usando su cuenta vía Gmail API).

- **RS-3.2:** Todo análisis se guarda en dos archivos JSON locales: `analisis.json` (detalle completo) y `resumen.json` (estadísticas agregadas). No se anonimizan datos porque el usuario es el propietario de la cuenta.

IV-D. RF-04: Visualización de Resúmenes y Estadísticas

El usuario accede al dashboard mediante la URL pública generada por ngrok.

- **RS-4.1:** Dashboard web servido por Flask que muestra en tiempo real la información almacenada en los archivos JSON.
- **RS-4.2:** Visualizaciones implementadas:
  1. Gráfico de barras con conteo total de correos Legítimo / Spam / Phishing.
  2. Tabla paginada y filtrable con los últimos correos analizados (asunto, remitente, fecha, clasificación y nivel de confianza).
  3. Estadísticas generales (total procesados, porcentaje de amenazas, último análisis).

V. Requisitos No Funcionales

Los requisitos no funcionales (RNF) definen las cualidades y restricciones del sistema. A continuación se presentan los que realmente se cumplieron en la implementación final.

V-A. Requisitos del Producto

- **RNF-01 (Eficiencia):** El tiempo de clasificación de un correo electrónico es inferior a 2 segundos en una instancia e2-micro de GCP
- **RNF-02 (Fiabilidad):** Disponibilidad superior al 99 % durante las pruebas continuas, con reinicio automático del contenedor Docker ante fallos inesperados.
- **RNF-03 (Usabilidad):** Interfaz web intuitiva que permite conectar la cuenta Gmail y visualizar resultados en menos de 3 clics, sin requerir conocimientos técnicos.
- **RNF-04 (Seguridad):** Los tokens OAuth 2.0 se almacenan cifrados mediante variables de entorno seguras. El sistema solo accede en modo lectura a la bandeja y no persiste contenido de correos más allá de los archivos JSON locales del propio usuario.

V-B. Requisitos de la Organización

- **RNF-05 (De Desarrollo):** Implementación completa en Python 3.10+ utilizando Flask como framework web y plantillas HTML/JavaScript estáticas, siguiendo estándares de código abierto y buenas prácticas.
- **RNF-06 (Operacionales):** El sistema opera exclusivamente en una instancia e2-micro gratuita de Google Cloud Platform, dentro de un contenedor Docker único, con exposición pública mediante túnel ngrok.
- **RNF-07 (Portabilidad):** Despliegue posible en cualquier máquina con Docker y Python 3.10+ (mínimo 2 GB RAM recomendados), sin dependencia de servicios pagos ni configuraciones propietarias.

V-C. Requisitos Externos

- **RNF-10 (Sostenibilidad):** El sistema opera completamente dentro de las cuotas gratuitas de Gmail API y de la instancia e2-micro always-free de Google Cloud Platform. El costo operativo total se mantiene por debajo de 1 USD/mes gracias al uso eficiente del disco persistente mínimo requerido para garantizar la disponibilidad continua de los datos procesados, cumpliendo con los objetivos de accesibilidad económica del proyecto.

VI. Arquitectura del Sistema

La arquitectura del sistema Chasky se ha diseñado para servir como un puente entre los requerimientos especificados y la implementación detallada, proporcionando una estructura robusta y escalable.

VI-A. Principios y Diseño Arquitectónico

El diseño se fundamenta en principios clave de la ingeniería de software para garantizar la calidad y mantenibilidad del sistema:

- **Organización en Capas:** Se establece una separación clara de responsabilidades entre la capa de presentación (interfaz de usuario), la lógica de negocio (procesamiento de correos) y el acceso a datos (interacción con la API de Gmail y persistencia).
- **Uso de Patrones Arquitectónicos:** La arquitectura adopta el patrón Modelo-Vista-Controlador (MVC) para la estructura del dashboard web y un esquema Cliente-Servidor para la comunicación con servicios externos como la API de Gmail.
- **Escalabilidad y Modularidad:** El diseño está concebido para permitir una evolución futura, facilitando la transición desde el prototipo actual hacia un sistema distribuido más complejo si fuese necesario.

VI-B. Vistas Arquitectónicas (Modelo 4+1)

Para describir la arquitectura desde múltiples perspectivas, se utiliza el modelo 4+1, asegurando que todos los aspectos del sistema, tanto estáticos como dinámicos, queden cubiertos.

VI-B.1. Vista Lógica: Esta vista define la estructura funcional del sistema. Se compone de los siguientes módulos principales:

- **Motor de Autenticación:** Encapsula toda la lógica de autorización y gestión de sesiones mediante el protocolo OAuth 2.0 para interactuar de forma segura con Gmail.
- **Módulo de Monitoreo:** Responsable de la adquisición de nuevos correos electrónicos desde la cuenta del usuario a través de la API de Gmail.
- **Módulo de Procesamiento:** Contiene la lógica para el análisis del contenido de los correos y la extracción de características.
- **Módulo de Persistencia:** Abstrae el almacenamiento y la recuperación de los resúmenes de correos generados, utilizando archivos en formato JSON.
- **Módulo de Visualización:** Gestiona la presentación de datos y estadísticas en el dashboard web interactivo.

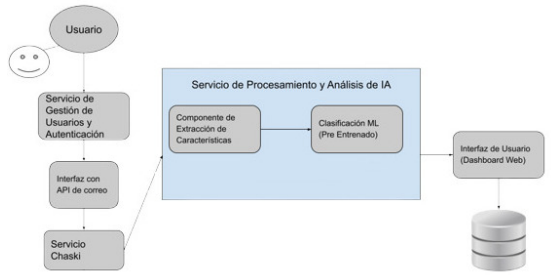

Fig. 1. Vista lógica del sistema Chasky, Arquitectura de Servicios

VI-B.2. Vista de Procesos: Esta vista se enfoca en el comportamiento dinámico del sistema y la comunicación entre procesos en tiempo de ejecución. El flujo principal es el análisis de un correo, que se inicia con la autenticación del usuario, procede con el monitoreo y procesamiento, y culmina con la generación y persistencia de un resumen. Describe también procesos concurrentes como la actualización periódica de las estadísticas del dashboard y el registro asíncrono de los resúmenes.

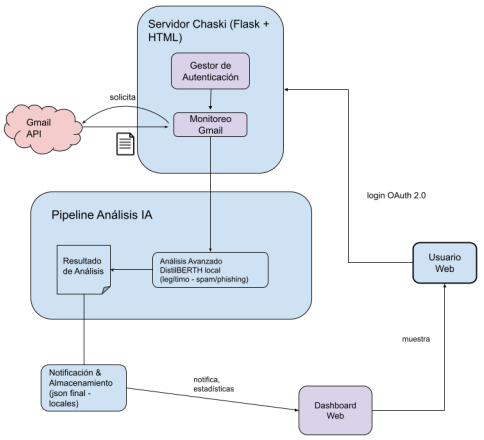

Fig. 2. Vista de procesos del sistema Chasky, mostrando el flujo de datos y la interacción dinámica entre los módulos principales.

VI-B.3. Vista de Desarrollo: Esta vista organiza el sistema en módulos de software para facilitar su construcción y mantenimiento. El proyecto se estructura en un **backend monolítico desarrollado en Python con el framework Flask**, el cual también se utiliza para implementar el frontend del dashboard. Un módulo adaptador específico gestiona la comunicación con la API de Gmail, desacoplando la lógica de negocio de la API externa. La persistencia de datos se maneja a través de la interacción con archivos JSON.

VI-B.4. Vista Física (Despliegue): Esta vista detalla la infraestructura de hardware y la distribución de los componentes de software. El sistema se despliega completamente en Google Cloud Platform (GCP) utilizando contenedores Docker. La topología consiste en:

- Un único servidor de aplicación ejecutado en una instancia e2-micro (capa gratuita) con 1 vCPU compartida y hasta 1 GB de RAM, que contiene todo el sistema Chasky (backend y frontend con Flask) dentro de un solo contenedor Docker.
- Un disco persistente HDD de 100 GB montado en la misma instancia para garantizar la persistencia de los archivos JSON entre reinicios.
- Acceso público al dashboard mediante túnel seguro ngrok, que proporciona conexión HTTPS sin necesidad de IP pública estática ni certificados propios.

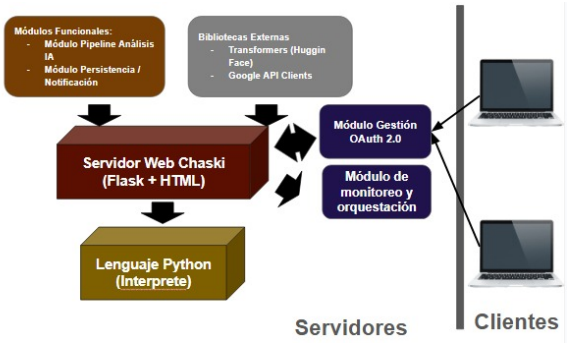

Fig. 3. Vista de desarrollo del sistema Chasky, con la asignación de tecnologías a cada componente.

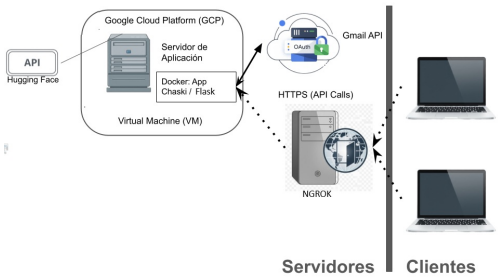

Fig. 4. Vista física del sistema Chasky, mostrando la distribución de componentes en la nube, los dispositivos de usuario y la comunicación con Gmail API.

VI-B.5. Vista de Casos de Uso (+1): Esta vista actúa como el elemento unificador que valida la arquitectura completa a través de escenarios de uso reales. Los casos de uso principales que el diseño debe satisfacer son: la autenticación de un usuario, el monitoreo y procesamiento de correos, la generación de resúmenes, y la visualización de datos en el dashboard. Cada uno de estos escenarios verifica que las otras cuatro vistas operan de manera cohesiva para cumplir los objetivos del proyecto.

VII. Modelos del Sistema

Para detallar el diseño interno del sistema y la interacción entre sus componentes, se utilizaron modelos estándar de la ingeniería de software, tal como se especifica en el documento de requerimientos del proyecto.

VII-A. Diseño Orientado a Objetos

Aunque la implementación final prioriza simplicidad y funcionamiento estable en recursos muy limitados, el sistema

se estructura mediante clases claras con responsabilidades bien definidas.

VII-A.1. Clases principales:

- **ChaskyEngine:** Clase principal que orquesta todo el sistema. Contiene instancias de GmailService, EmailAnalyzer y NotificationService, además del control del último timestamp procesado.
- **GmailService:** Encapsula la autenticación OAuth 2.0 y todas las operaciones con la Gmail API (listar mensajes, obtener mensaje completo y enviar alertas).
- **EmailAnalyzer:** Carga el modelo DistilBERT y su tokenizer una sola vez en memoria. Expone el método `analyze(email: dict)` → `AnalysisResult`.
- **NotificationService:** Responsable exclusivo del envío de alertas por correo electrónico cuando se detecta phishing o spam de alta confianza.
- **AnalysisResult:** Clase de datos (dataclass) que almacena el resultado del análisis: predicción, confianza, nivel de riesgo (1–5), indicadores y timestamp.

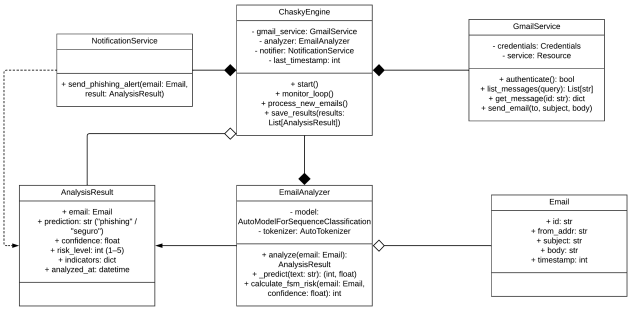

Fig. 5. Diagrama UML, mostrando las clases y atributos que componen al proyecto Chasky

VII-A.2. Diagrama de Secuencia: El flujo principal de operación es el siguiente:

1. Al iniciar el contenedor se crea una instancia de `ChaskyEngine`.
2. `ChaskyEngine.start()` lanza un hilo infinito que ejecuta `monitor_loop()`.
3. Periódicamente, `GmailService` obtiene los correos nuevos.
4. Cada correo se envía a `EmailAnalyzer.analyze()`.
5. El resultado se guarda en archivos JSON y, si es amenaza, se dispara `NotificationService.send_phishing_alert()`.
6. El dashboard web (Flask) lee los JSON mediante polling y muestra los datos actualizados.

VIII. Conclusión

La especificación de requerimientos desarrollada para el sistema "Chasky" ha constituido un paso fundamental en la planificación y el diseño de una herramienta orientada a la gestión y análisis de correos electrónicos. La definición clara de los requerimientos funcionales ha permitido delinear las capacidades esenciales del sistema, como la autenticación segura, el análisis automático de correos, la generación de resúmenes y la visualización de datos en un tablero web.

De manera complementaria, los requerimientos no funcionales han asegurado que el sistema no solo cumpla sus objetivos, sino que lo haga de una forma eficiente, fiable, usable y en conformidad con la normativa vigente en materia de protección de datos personales. En conjunto, esta especificación ha sentado las bases para la implementación de un prototipo que aporta una solución con impacto social, fortaleciendo la ciberseguridad del usuario en un entorno controlado.

Referencias

Albán, F., Urvina, M., & Andrade, R. O. (2022). Analysis and Design of a Predictive Model for Phishing Detection Based on Url and Email Corpus; Análisis y Diseño de un Modelo Predictivo para Detección de Phishing Basado en Url y Corpus del Correo Electrónico [Cited by: 0; All Open Access; Gold Open Access]. *Revista Politecnica*, *50*(3), 27-42. https://doi.org/10.33333/rp.vol50n3.03

Alhuzali, A., Alloqmani, A., AlJabri, M. I., & Alharbi, F. (2025). In-Depth Analysis of Phishing Email Detection: Evaluating the Performance of Machine Learning and Deep Learning Models Across Multiple Datasets [Cited by: 4; All Open Access; Gold Open Access]. *Applied Sciences (Switzerland)*, *15*(6). https://doi.org/10.3390/app15063396

Butt, U. A., Amin, R., Aldabbas, H., Mohan, S., Alouffi, B. M., & Ahmadian, A. (2023). Cloud-based email phishing attack using machine and deep learning algorithm [Cited by: 80; All Open Access; Green Accepted Open Access; Green Open Access; Hybrid Gold Open Access]. *Complex and Intelligent Systems*, *9*(3), 3043-3070. https://doi.org/10.1007/s40747-022-00760-3

Fatima, R., Fareed, M. S., Ullah, S., Ahmad, G., & Mahmood, S. (2024). An Optimized Approach for Detection and Classification of Spam Email's Using Ensemble Methods [Cited by: 5; All Open Access; Hybrid Gold Open Access]. *Wireless Personal Communications*, *139*(1), 347-373. https://doi.org/10.1007/s11277-024-11628-9

Johnson, C., Khadka, B., Basnet, R. B., & Doleck, T. (2020). Towards detecting and classifying malicious urls using deep learning [Cited by: 75]. *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications*, *11*(4), 31-48. https://doi.org/10.22667/JOWUA.2020.12.31.031

Meléndez, R., Ptaszynski, M. E., & Masui, F. (2024). Comparative Investigation of Traditional Machine-Learning Models and Transformer Models for Phishing Email Detection [Cited by: 6; All Open Access; Gold Open Access]. *Electronics (Switzerland)*, *13*(24). https://doi.org/10.3390/electronics13244877